Zadanie č. 3: Konvolučné siete a prenos vedomostí

- Dokumentácia

Meno:Martin Kasznár **Krúžok:piatok od 10:00 do 12:00**

Úvod

Zadanie súvisí s tým, že de facto* je nám predstavený dataset o dĺžke 6007 obrázkov s 3 farebnými kanálmi RGB a k nemu prislúchajúca tabuľka-navigátor o dĺžke 6007 riadkov s 19 stĺpcami s rôznymi typmi hodnôt a 1 cieľovým stĺpcom s hodnotou **Irradiance** žiarivosť oblohy. **Popísaný účel tabuľky k tomuto zadaniu je navigovať kde sú obrázky a informovať aká im prináleží Irradiance**. Necieľové stĺpce tabuľky popisujú množinu meteorologických meraných metrík súvisiacich so zhotovovaním prináležiaceho obrázku. Ide o to spraviť odhadovací regresný nástroj, ktorý na vstup obrázok odpovie výstupom odhadovaná žiarivosť oblohy. Stĺpec DateTime->Year,Month,Hour (label/onehot) pri EDA, inde ostáva nezmenený a stĺpec PictureName si zachová výlučne navigačný charakter.
K tomu máme pristúpiť tak, že treba po 1.) **vykonať jednoduchšiu EDA a opatrne vykonať delenie na časti train,valid,test, fragmentovaného datasetu s dvoma typmi zložiek,** po 2.) **zostaviť a predvádzať vlastný CNN model,** po 4.) **zostaviť a predvádzať čiastočne odmrazený stiahnutý CNN model natrénovaný na ImageNet datasete** a bonus.) **predvádzať aj na vlastnoručne zostrojených obrázkoch**. Bod 3.) zhlukovanie bol vynechaný.
*dataset je predstavený fragmentovaný a jednou z úloh je ho zložiť

– Analyzujte dostupné tabuľkové dáta a vizualizujte vzťahy medzi jednotlivými premennými vzhľadom na cieľovú premennú. Keďže máme k dispozícii obrazové aj tabuľkové dáta, rozdeľte ich na tri časti – trénovaciu, validačnú a testovaciu množinu (X-obrázky, Y-Stĺpec Irradiance). Pri rozdeľovaní zabezpečte, aby boli dáta správne spárované a vzájomne konzistentné. **2b**

1.1 Skladanie datasetu

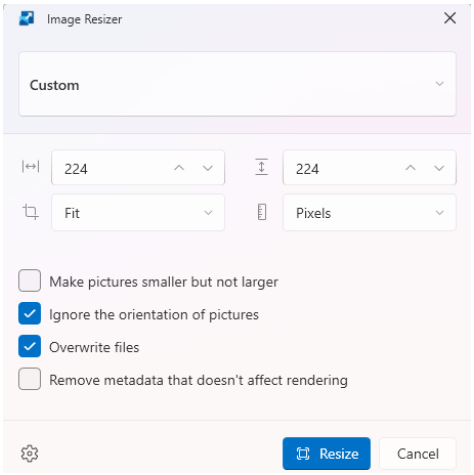
Získali sme nasledovnú štruktúru:

```
Solar_data
->01
->->original
->->->Alpnach_20180101_07-15-00_ExposureStack_Image_10_image.png
->->->…
->->out_data.csv
->04
->->original
->->->...
->->out_data.csv
->07
->->original
->->->...
->->out_data.csv
->10
->->original
->->->...
->->out_data.csv
```

pričom by sa lepšie pracovalo so štruktúrou:
Root
->ins
->->in_big_png
->->->...
->->in_own_png
->->->...
->->in_small_png
->->->…
->92414_umint_prog_zad3.py
->navig.csv
kde platí, že a) **navig.csv** sú spojené všetky 4 out_data.csv dokopy, lebo majú spoločné stĺpce a dáta v nich sa neprelínajú; že b) **in_big_png** sú spojené všetky 4 priečinky original dokopy z rovnakého dôvodu; že c) **in_small_png** sú zmenšené obrázky z in_big_png na arbitrárnu veľkosť a že d) **in_own_png** sú vlastnoručne zostrojené obrázky.
Na bod a) poslúži tento dočasný kód v prítomnosti premenovaných out_data.csv podľa mesačnej akosti:

```
import pandas as pd
files = ["in_1.csv", "in_4.csv", "in_7.csv", "in_10.csv"]
dfs = []
for f in files:
    df = pd.read_csv(f, skip_blank_lines=True)
    dfs.append(df)
merged = pd.concat(dfs, ignore_index=True)
merged.to_csv("in.csv", index=False)
```

Bod b) súvisí s triviálnou drag and drop operáciou.
Na bod c) bol využitý stiahnuteľný nástroj Microsoft PowerToys->skopírovať všetky veľké obrázky do priečinku s malými->označiť všetky obrázky Ctrl+A->RMB->Resize with Image Resizer->tieto nastavenia:



Bod d) šiel obdobne ako bod c), ale zvoliť možnosť **Stretch!!!**, vždy musí byť zachovaná veľkosť 224x224 a mobillné telefóny nefotia štvorcové fotografie.

1.2 EDA

Negrafická analýza

```
>>> df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
Irradiance	6007.0	299.049842	2.824998e+02	0.060000	56.550000	198.290000	489.985000	1136.560000
IrradianceNotCompensated	6007.0	298.687212	2.820621e+02	0.060000	56.530000	198.150000	489.540000	1134.590000
BodyTemperature	6007.0	20.526830	7.954458e+00	1.740000	13.850000	21.030000	26.780000	41.000000
RelativeHumidity	6007.0	3.166621	1.636661e+00	0.000000	1.860000	3.450000	4.480000	6.830000
HumidityTemp	6007.0	21.954931	7.925695e+00	3.210000	15.320000	22.470000	28.180000	42.390000
Pressure	6007.0	1027.791805	2.797024e+01	958.531250	1007.109375	1026.875000	1050.625000	1100.562500
PressureAvg	6007.0	1027.783034	2.797346e+01	958.687500	1007.125000	1026.937500	1050.656250	1100.500000
PressureTemp	6007.0	20.941335	8.258914e+00	1.343750	13.986328	21.437500	27.419922	42.269531
PressureTempAvg	6007.0	20.939610	8.259061e+00	1.429688	13.988281	21.457031	27.437500	42.269531
TiltAngle	6007.0	0.126802	6.747907e-02	0.000000	0.090000	0.120000	0.150000	1.600000
TiltAngleAvg	6007.0	0.109098	2.187936e-02	0.050000	0.090000	0.110000	0.130000	0.180000
FanSpeed	6007.0	9075.146496	1.158266e+02	8780.000000	8982.000000	9078.000000	9161.500000	9352.000000
HeaterCurrent	6007.0	0.329238	1.916442e-03	0.322000	0.328000	0.329000	0.331000	0.336000
FanCurrent	6007.0	0.067302	1.035614e-03	0.064000	0.067000	0.067000	0.068000	0.072000
SunLatitude	6007.0	46.935881	7.106019e-15	46.935881	46.935881	46.935881	46.935881	46.935881
SunLongitude	6007.0	8.275840	0.000000e+00	8.275840	8.275840	8.275840	8.275840	8.275840
SunAzimuth	6007.0	180.276015	6.484813e+01	55.453051	125.413340	180.366639	235.123478	304.240827
SunZenith	6007.0	27.241519	1.773102e+01	-0.263644	13.054332	23.996333	40.250652	66.160245
Year	6007.0	2018.000000	0.000000e+00	2018.000000	2018.000000	2018.000000	2018.000000	2018.000000
Month	6007.0	5.741468	3.097524e+00	1.000000	4.000000	7.000000	7.000000	10.000000
Hour	6007.0	12.885134	3.777899e+00	5.000000	10.000000	13.000000	16.000000	21.000000

účelom tohto príkazu je v **min** a **max** prezradiť koľko dát by napr. outlier manažment ukrojil (**v tomto zadaní o.m. nie je použitý**), v **25%** a **75%** ak by bol použitý IQR outlier manažment, tak aké by boli percentily rozhodujúce o ukrojení, **50%** je percentil 50, **mean** je stredná hodnota, smerodajná odchylka **std** prezrádza ako veľmi sú dáta rozptýlené od **meanu**, **count** prezrádza či majú dáta predpokladaný tvar

```
>>> df.head().T
```

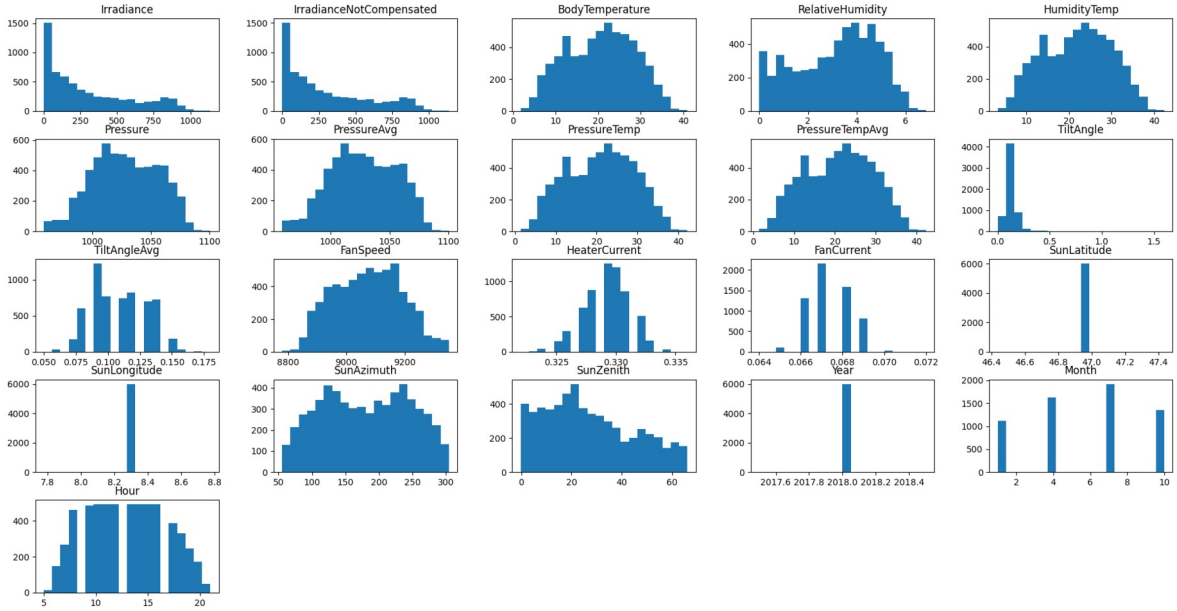
	0	1	2	3	4
PictureName	Alpnach_20180101_07-15-00_...	Alpnach_20180101_07-30-00_...	Alpnach_20180101_07-45-00_...	Alpnach_20180101_08-00-00_...	Alpnach_20180101_08-15-00_...
Irradiance	3.44	10.88	16.4	17.5	19.37
IrradianceNotCompensated	3.43	10.84	16.36	17.46	19.32
BodyTemperature	7.28	7.14	7.66	7.63	7.59
RelativeHumidity	0.08	0.07	0.18	0.15	0.15
HumidityTemp	8.74	8.63	9.16	9.11	9.06
Pressure	992.78125	992.28125	994.09375	993.875	993.8125
PressureAvg	992.8125	992.46875	994.34375	994.15625	993.90625
PressureTemp	7.214844	7.070312	7.660156	7.648438	7.625
PressureTempAvg	7.253906	7.109375	7.660156	7.628906	7.53125
TiltAngle	0.08	0.12	0.17	0.1	0.06
TiltAngleAvg	0.08	0.08	0.08	0.08	0.08
FanSpeed	9228	9232	9221	9225	9227
HeaterCurrent	0.328	0.328	0.327	0.326	0.326
FanCurrent	0.069	0.069	0.069	0.069	0.069
SunLatitude	46.935881	46.935881	46.935881	46.935881	46.935881
SunLongitude	8.27584	8.27584	8.27584	8.27584	8.27584
SunAzimuth	124.282567	127.035979	129.847232	132.721094	135.661756
SunZenith	0.081828	1.914794	3.813112	5.683627	7.488597
Year	2018	2018	2018	2018	2018
Month	1	1	1	1	1
Hour	8	8	8	9	9

vidíme náhľad do prvých 5 dát datasetu po cleaningu, dáta prezrádzajú adekvátny cast a stav premennej Month a Hour či sú label encoded alebo onehot encoded, lebo rôzne operácie si žiadajú ich rôzny stav (sú cyklické ordinály)

```
>>> df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6007 entries, 0 to 6006
Data columns (total 22 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   PictureName              6007 non-null   object
 1   Irradiance               6007 non-null   float64
 2   IrradianceNotCompensated 6007 non-null   float64
 3   BodyTemperature          6007 non-null   float64
 4   RelativeHumidity         6007 non-null   float64
 5   HumidityTemp             6007 non-null   float64
 6   Pressure                 6007 non-null   float64
 7   PressureAvg              6007 non-null   float64
 8   PressureTemp             6007 non-null   float64
 9   PressureTempAvg          6007 non-null   float64
 10  TiltAngle                6007 non-null   float64
 11  TiltAngleAvg             6007 non-null   float64
 12  FanSpeed                 6007 non-null   int64
 13  HeaterCurrent            6007 non-null   float64
 14  FanCurrent               6007 non-null   float64
 15  SunLatitude              6007 non-null   float64
 16  SunLongitude             6007 non-null   float64
 17  SunAzimuth               6007 non-null   float64
 18  SunZenith                6007 non-null   float64
 19  Year                     6007 non-null   int32
 20  Month                    6007 non-null   int32
 21  Hour                     6007 non-null   int32
dtypes: float64(17), int32(3), int64(1), object(1)
memory usage: 962.2+ KB
>>>
```

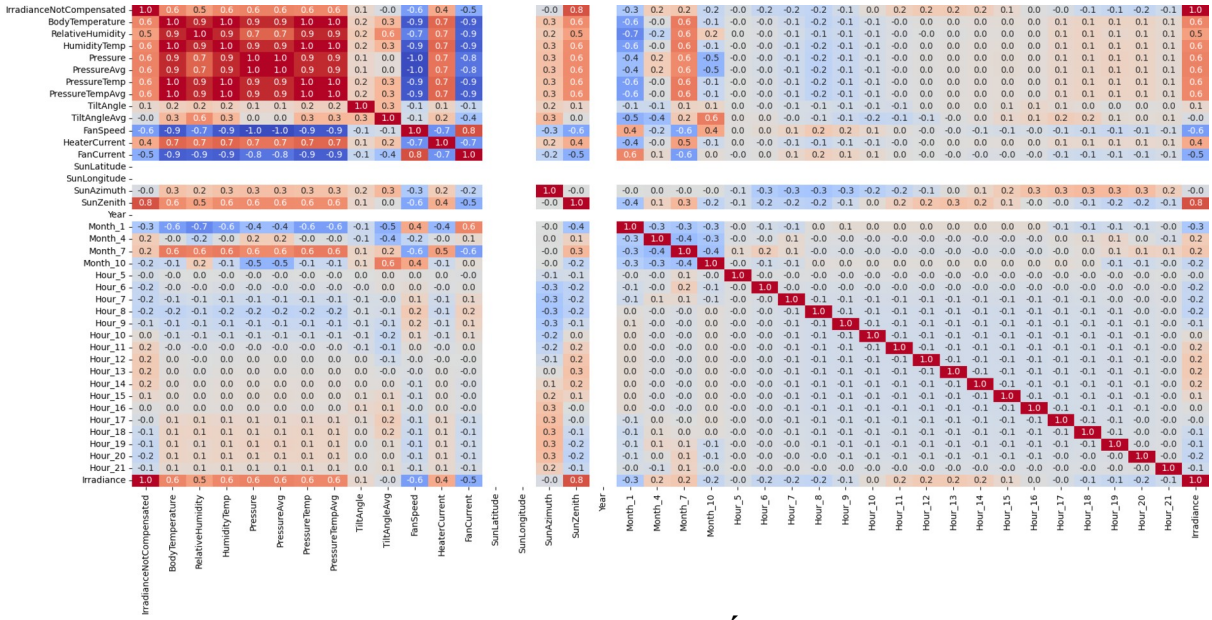
debugovacie info oznamujúce napr. casty premenných

Univariátna mapa



na grafoch univariátnej analýzy vidíme, ktoré typy stĺpcov sme dobre zdokumentovali a podľa výsledkov prieskumu bivariátnych korelácií budeme vedieť či je potreba získať viac dát o tipe na silne korelovanú premennú priamo s cieľovou premennou

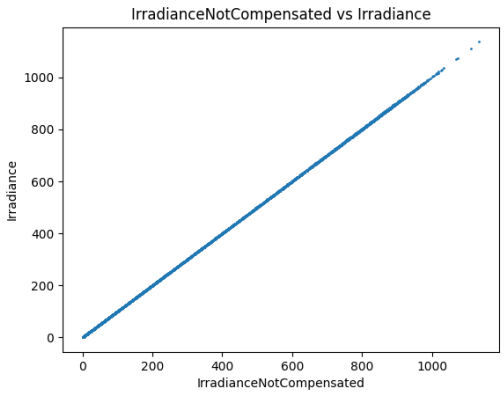
Bivariátna mapa



na grafe bivariátnej mapy vidíme, ktoré typy **všetkých stĺpcov** sú vzájomne záporne, nulovo či kladne korelované, toto má slúžiť ako **mapa na nájdenie prieskumuhodných bivariátnych korelácií**, ktoré sú ešte preverené grafom proti cieľovej premennej

Prieskum bivariátnych korelácií

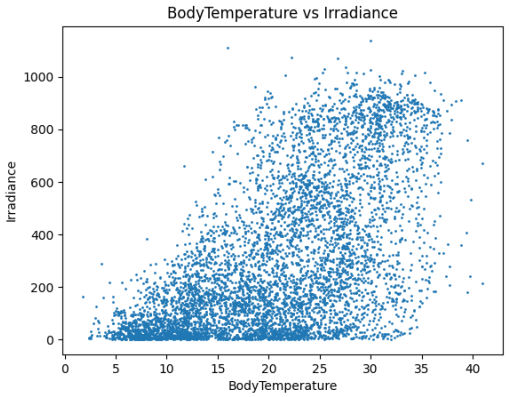
a) Irradiance<->IrradianceNotCompensated



K tomuto obrázku **porovnania nekompenzovanej Irradiance** vs. **kompenzovanej Irradiance**, možno povedať to, že pri priblížení vidieť minimálne rozdiely, ale tieto premenné sú rôzne, inak by tvorili rovnú čiaru.

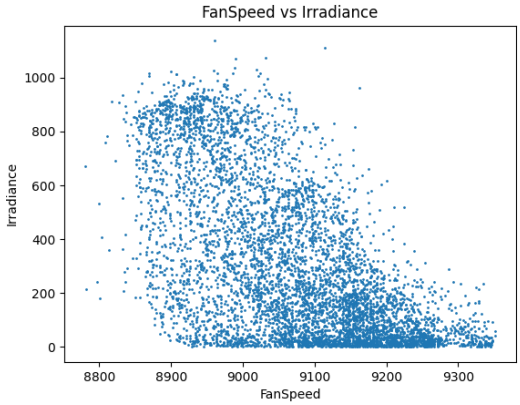
Korelácia je najsilnejšia možná: 1.0.

b) Irradiance<->BodyTemperature



K tomuto obrázku porovnania **teploty kamery** vs. **Irradiance** možno povedať to, že svietivá obloha koreluje so zahriatejšou kamerou (zjavne ju zahrialo prostredie, ktoré je teplé za svetla, chladné za tmy).
Korelácia je stredne vysoká: 0.6

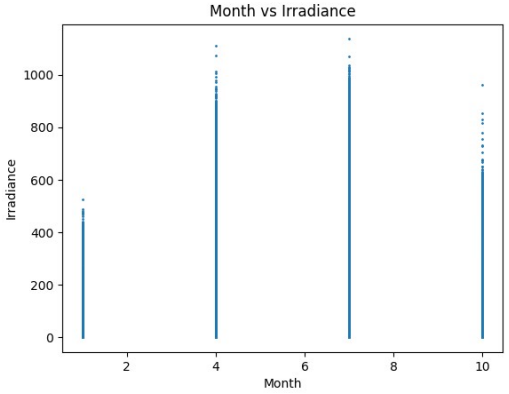
c) Irradiance<->FanSpeed



Tu sa porovnáva **rýchlosť turbíny vedľa kamery** vs. **Irradiance** a možno povedať to, že veternejšie počasie **negatívne** koreluje so svietivou oblohou, ale toto ani nie je zaujímavá informácia, keďže FanSpeed veľmi vysoko **negatívne** koreluje s BodyTemperature (-0.9).

Korelácia je stredne vysoká **negatívna**: -0.6

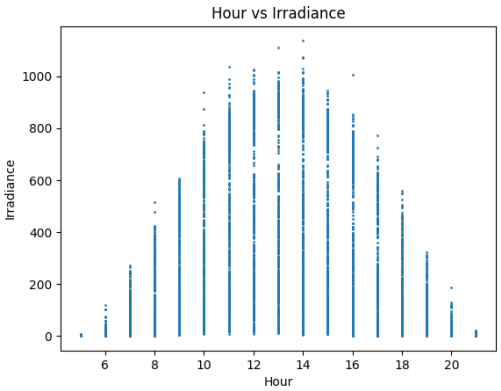
d) Irradiance<->Month



Tu porovnávame **mesiac** vs. **Irradiance** a možno by bolo možné usúdiť, že v lete sú dni svietivé častejšie pre akúkoľvek hodinu ako v zime, no závisí to aj od vyváženosti datasetu v univariátnej mape. Toto je ale aj triviálne pravda, keďže v lete sú dni dlhšie.

Korelácia s januárom:aprílom:júlom:októbrom:
-0.3:0.2:0.2:-0.2

e) Irradiance<->Hour



Tu porovnávame **hodinu** vs. **Irradiance** a je úplne zjavné, že v hodinu bližšiu k poludniu je obloha najsvietivejšia a v hodinu bližšiu k polnoci zase najmenej.

Korelácia s hodinami: -0.2:…:0:…:0.2:…:0:...-0.2

1.3 train, valid, test split

```python
class IrradianceImageDataset():
    def __init__(self):
        self.transform = T.Compose([
            T.ToTensor()
        ])

        self.image_names = [
            fname for fname in PNI_DICT.keys()
            if os.path.isfile(os.path.join(IMAGES_FOLDER, fname))
        ]
    def __len__(self):
        return len(self.image_names)
    def __getitem__(self, idx):
        img_name = self.image_names[idx]
        img_path = os.path.join(IMAGES_FOLDER, img_name)

        image = Image.open(img_path).convert('RGB')

        image = self.transform(image)

        target = float(PNI_DICT[img_name])
        target = torch.tensor([target], dtype=torch.float32)

        return image, target
```

Na týchto obrázkoch je zdokumentované to, že split prebieha tak, že najprv je definovaná celá trieda tohto datasetu IrradianceImageDataset, ktorá má časť súvisiacu s jej transformáciou (<0,255> -> <0,1>), pozná mená svojich obrázkov, svoju dĺžku, vie indexovať v obrázkoch, **obrázok pri pokuse o prečítanie skonvertuje do RGB, následne normalizuje kanály do <0,1>**, následne predpripraví obrázku prináležiaci Irradiance a **vie na výzvu odpovedať párom obrázok, Irradiance.**

```python
dataset = IrradianceImageDataset()
rat_tn, rat_vd = 0.7, 0.15
n_full = len(dataset)
n_tn = int(rat_tn * n_full)
n_vd  = int(rat_vd * n_full)
n_tt  = n_full - n_tn - n_vd
print('Split plan:',n_full,'->', [n_tn, n_vd, n_tt])
generator = torch.Generator().manual_seed(42)
train_ds, val_ds, test_ds = torch.utils.data.random_split(
    dataset,
    [n_tn, n_vd, n_tt],
    generator=generator
)

batch_size = 32
train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True)
val_loader   = DataLoader(val_ds,   batch_size=batch_size, shuffle=False)
test_loader  = DataLoader(test_ds,  batch_size=batch_size, shuffle=False)
```

Pomer train setu:valid setu:test setu nastavíme tak, že zadáme napr. **rat_tn=0.7** (train set bude 70% setu), **rat_vd=0.15** (valid set bude 15% setu) a program si sám vyvodí, že (test set bude 100-70-15=15%). Zadávať sane hodnoty rat_tn a rat_vd z (0,1) a zároveň aby ich súčet bol z (0,1).

Tu vieme zmeniť aj **batch_size=32** na iné hodnoty.

Na obrázku vidieť zhotovovanie generátoru dát, na 1. zvýraznenej vete strany vidieť, že normalizácia je implementovaná pri pokuse o prečítanie obrázku a veľkosti obrázkov sme zmenili z 1200x1200->224x224, ktorá je adekvátna pre drvivú väčšinu sťahovateľných modelov, ešte v časti 1.1. Je vidieť správne použitie shuffle. x,y odpoveď Map-style datasetu je pár obrázok, Irradiance.

Motivácia delenia je v tom, že po epochách sa na trénovaciu množinu nahadzuje zlepšujúca sa krivka, ale aby nebola nevšeobecná na nevidený obrázok ju uvoľňujeme validačnou množinou. Testovacia množina slúži na jediné otestovanie a toto má simulovať náhodný obrázok, ktorý už bol v produkcii nahratý a chceme otestovať ako veľmi je svietivý, lebo testovacia množina v trénovaní nie je videná modelom ani nepriamo cez tresty.

2 Zostavenie a predvádzanie vlastného CNN modelu

- Pre trénovaciu, validačnú a testovaciu množinu, si pripravte generátor dát, nezabudnite dáta normalizovať a vhodne im zmeniť veľkosť - pozor na zamiešanie (shuffle) dát pri použití generátorov. POZOR: vstup je obrázok a výstup z hodnota z tabuľkových dát. **0.5b**
- Navrhnite sieť s aspoň 2 konvolučnými vrstvami **a** využite niektoré z regularizačných alebo normalizačných techník aby ste zabránili pretrénovaniu(aspoň 2). Skúste niekoľko (aspoň 5) cielených konfigurácií hyperparametrov, všetky úspešnosti**(MSE, MAE, RMSE a R2)** pre **trénovaciu** aj **testovaciu** množinu zobrazte **v 1 tabuľke**. Nezabudnite aj na vykreslenie reziduálov (**trénovacej** aj **testovacej**). **2b**
- Nezabúdajte na dobré praktiky zo zadania 1 a 2 - sledujte a zobrazte priebehy trénovania, poriadne evaluujte na **trénovacej** a **testovacej** množine (analyzujte reziduály, môžete vykreslit aj predikované vs skutočné hodnoty alebo iné).

V tabuľke uveďte úspešnosti**(MSE, MAE, RMSE a R2)** pre **trénovaciu** aj **testovaciu** množinu, grafy a reziduály stačí zobraziť pre najlepšie trénovanie. **1b**

Našou úlohou je navrhnúť **vlastnú CNN sieť** s **2 a viac konvolučnými vrstvami**, použiť **2 a viac reg./norm. techník**, skúšať **5 rôznych konfigov** hyperparametrov->pre ne **MSE, MAE, RMSE, R2** pre train/test v jednej tabuľke.

Vyhodnotiť ktorý je **najlepší model** a pripojiť k nemu **priebeh train/test loss**, **train/test reziduály, train/test predikované vs. skutočné hodnoty**.

Predstavenie našej CNN:

```
class CNNRegressor(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 16, kernel_size=3, padding=1),
            nn.BatchNorm2d(16),
            nn.ReLU(),
            nn.MaxPool2d(2), # 224 -> 112

            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2), # 112 -> 56

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2), # 56 -> 28
        )
        self.regressor = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 28 * 28, 128),
            nn.ReLU(),
            nn.Dropout(p=0.5),
            nn.Linear(128, 1)
        )
    def forward(self, x):
        x = self.features(x)
        x = self.regressor(x)
        return x
```

Naša CNN používa 3 konvolučné vrstvy, 1 normalizačnú techniku BatchNorm v CNN, kde normalizujeme aktivácie mini batchov, aby sme sa vyhli exploding a vanishing gradientu v C časti CNN. Používame 1 regularizačnú techniku Dropout v NN, kde pri forward passe zlomok aktivácií nastavíme na 0 a to podporí to, že NN časť CNN nebude memorovať, ale bude chápať v širokých súvislostiach problém, keďže sa nemôže spoľahnúť na špecifické neuróny.

epochs=100, **learning rate=0.001**, **patience=4**, t.j. je implementovaný EarlyStopping proti pretrénovaniu, podtrénovaniu, ak 4x za sebou nezlepšíme validačný MSE Loss.

Následne budeme experimentovať s:

3x learning rate, 1x vynechaním konvolučnej vrstvy, 1x znížením počtu neurónov poslednej vrstvy NN časti CNN

Víťaza stanovíme podľa metriky testovacej MSE Loss, keďže toto značí potenciálnu všeobecnosť modelu.

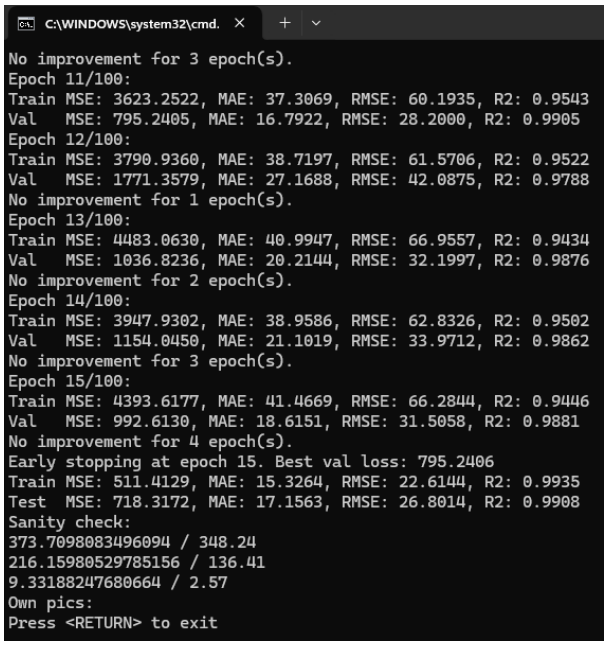
Skúsme tieto experimenty:

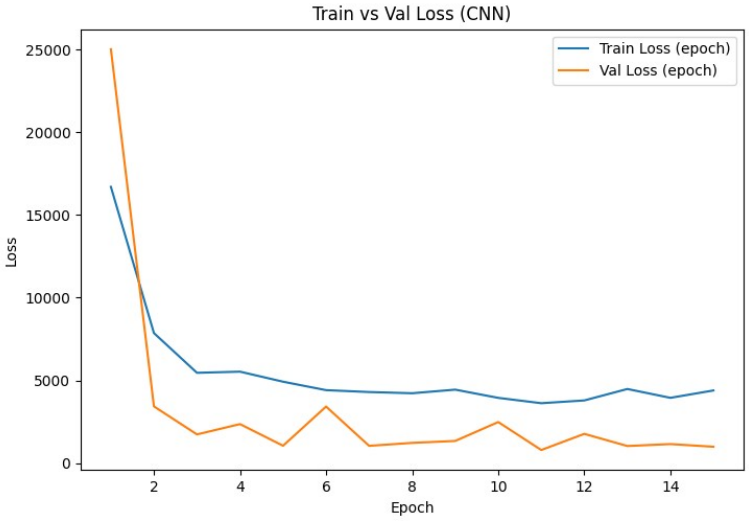
learning rate	C časť	NN časť	Train MSE	Train MAE	Train RMSE	Train R2	Test MSE	Test MAE	Test RMSE	Test R2
0.005	3->16->32->64	64x28^2->128->1	2126.4187	28.5321	46.1131	0.9732	2427.0034	29.5190	49.2646	0.9689
0.001	3->16->32->64	64x28^2->128->1	558.2826	14.9761	23.6280	0.9930	806.9048 👑	16.5603	28.4061	0.9897
0.0005	3->16->32->64	64x28^2->128->1	607.3122	16.3239	24.6437	0.9923	864.1704	18.4642	29.3968	0.9889
0.001	**3->16->32**	**32x56^2->128->1**	526.2334	15.0987	22.9398	0.9934	855.2717	17.2683	29.2450	0.9890
0.001	3->16->32->64	**64x28^2->64->1**	1030.1091	21.4801	32.0953	0.9870	1239.2869	23.1118	35.2035	0.9841

Experiment bol užitočný aj na stanovenie toho či oproti **normálu (2. riadok)** nepoužívame príliš výkonný nástroj na príliš malý problém. **Learning rate normálu sa ukazuje ako optimálny** keď je 0.001 (najnižšie Test MSE). **Na normáli by sa mohlo zmeniť to, že by sme mohli vynechať 1 konvolučnú vrstvu a neuškodilo by mu to výrazne (Test MSE +-50). To či znížiť počet neurónov poslednej vrstvy NN časti CNN je hraničné (Test MSE +-400).**

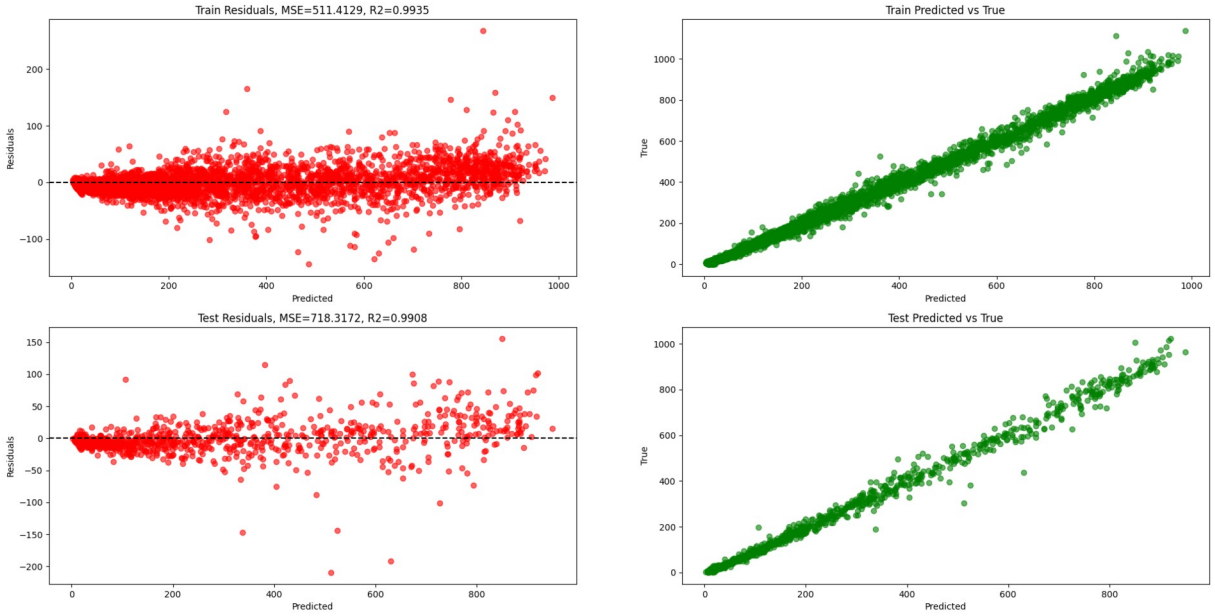
Vyhodnotenie

Podľa Test MSE vyhráva normál z 2. riadku s najnižším Test MSE 806.904. Opätovne spustíme program s týmito hyperparametrami a prikladáme snímky obrazovky:





Vidíme, že sieť nadobudla Test MSE 718.3172, blízke ku 806.904 a vidíme spôsob zaznamenávania metrík do tabuľky. Tiež používame Sanity check, kde používame pripravený model priamo na obrázkoch na odhad svietivosti a kontrastujeme s reálnymi hodotami. Uvádzame graf Train vs Val Loss. Acc by nemal zmysel pri regresnom probléme. Na grafe vidíme aspoň to, že EarlyStopping zastavil pretrénovavanie bez zlepšenia bez toho, aby trénovanie skončil moc skoro, čím by spôsobil potenciálne podtrénovanie.



Uvádzame reziduály aj porovnanie predikovanej svietivosti vs. reálna svietivosť na najlepšom vlastnom CNN modeli na train aj test sete. Na test set reziduáloch vidieť, že miera tmy je ľahšie odhadnuteľná než miera svetla, čo potvrdzuje aj test set porovnanie predikcia vs. realita. Reziduály môžu slúžiť na zistenie toho aké extrémne zlé predikcie môžu byť hoci by z celku vyplývalo adekvátne R2 skóre.

4 Zostavenie a predvádzanie stiahnutého CNN modelu

– **Možnosť 2** Dotrénujte posledné vrstvy ImageNet siete (ostatné budú zamrazené!).

Body budú udelené nasledovne:

– Natrénovanie modelu **2b**

– Zobrazenie priebehov trénovania (povinne, ak je to možné - t.j. neurónové siete v oboch prípadoch), reziduály, úspešnosti na **trénovacej** a **testovacej** množine + **analýza**: Kde sa model mýli? Prečo? **2b**

Našou úlohou je **spojazdniť stiahnutú CNN sieť** predtrénovanú na ImageNet datasete **v súvislosti s funkcionalitou zamrazených vrstiev**, vyskúšať 1 konfig hyperparametrov, preň->**MSE, MAE, RMSE, R2** pre train/test v jednej tabuľke.

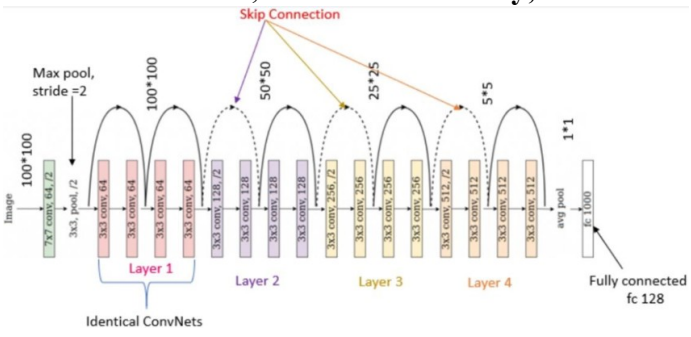
Vyhodnotiť **model** a pripojiť k nemu **priebeh train/test loss**, **train/test reziduály, train/test predikované vs. skutočné hodnoty**.

Predstavenie ľahko modifikovaného ResNet18:

```
class ResNet18Regressor(nn.Module):
    def __init__(self):
        super().__init__()
        weights = models.ResNet18_Weights.IMAGENET1K_V1
        regressor = models.resnet18(weights=weights)

        for param in regressor.parameters():
            param.requires_grad = False
        for param in regressor.layer4.parameters():
            param.requires_grad = True
        for param in regressor.fc.parameters():
            param.requires_grad = True

        in_features = regressor.fc.in_features
        regressor.fc = nn.Linear(in_features, 1)
        self.regressor = regressor
    def forward(self, x):
        return self.regressor(x)
```



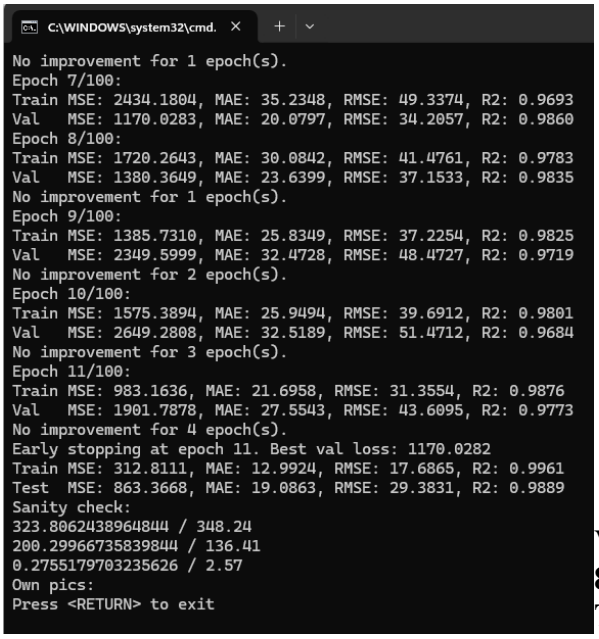
V prvom rade budeme používať: **epochs 100**, **learning rate 0.001**, **patience 4**. K sieti toľko, že si stiahneme nemodifikovaný ResNet18, následne zamrazíme

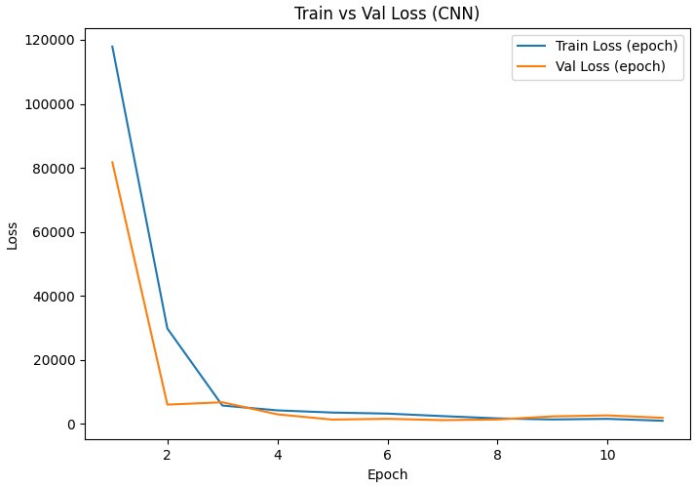
trénovanie všetkých vrstiev a odmrazíme layer 4 a fc z obrázku vpravo. Koniec CNN doladíme na regresiu (1 výstup). Trénujeme len odmrazené.

Vyhodnotenie:

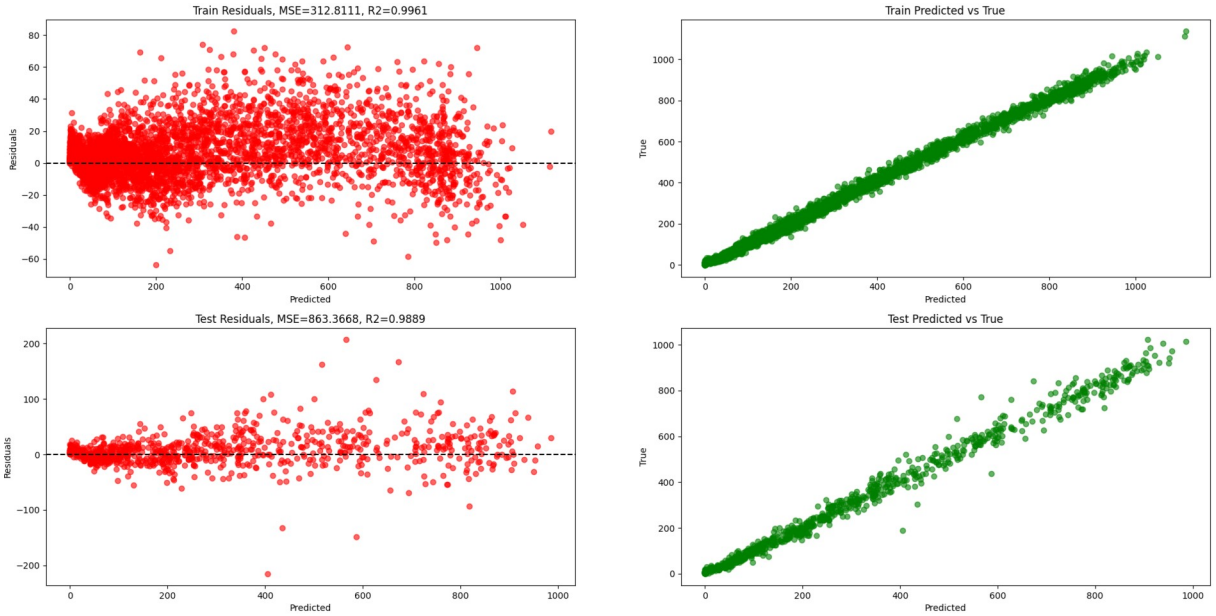
Model má samozrejme značne odlišnú architektúru siete. **Stále však používame epochs, learning rate** a **patience víťaznej CNN zo sekcie 2**.

learning rate	C časť	NN časť	Train MSE	Train MAE	Train RMSE	Train R2	Test MSE	Test MAE	Test RMSE	Test R2
0.001	R18: l4,fc ❄	R18: l4,fc ❄	312.8111	12.9924	17.6865	0.9961	863.3668 🤷	19.0863	29.3831	0.9889





Vidíme, že **ResNet18 sieť nadobudla Test MSE 863.3668** stopovo horšie než **vlastná CNN sieť s Test MSE 718.3172**. Uvádzame graf Train vs Val Loss. Na grafe vidíme aspoň to, že EarlyStopping zastavil pretrénovavanie bez zlepšenia bez toho, aby trénovanie skončil moc skoro, čím by spôsobil potenciálne podtrénovanie.



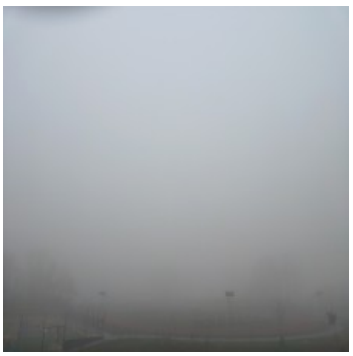
Uvádzame reziduály aj porovnanie predikovanej svietivosti vs. reálna svietivosť na najlepšom ResNet18 CNN modeli na train aj test sete. Na test set reziduáloch opäť vidieť, že miera tmy je ľahšie odhadnuteľná než miera svetla, čo potvrdzuje aj test set porovnanie predikcia vs. realita.

Bonus

Predvádzame vlastný CNN model vo víťaznej konfigurácii a ResNet18 model s modifikáciou zmrazenia vo svojej jedinej popísanej konfigurácii. Horné snímky boli zhotovené v okrese BA, dolné snímky boli zhotovené v okrese TO.**čas (vlastný model preds, R18+l4,fc❄ preds)**



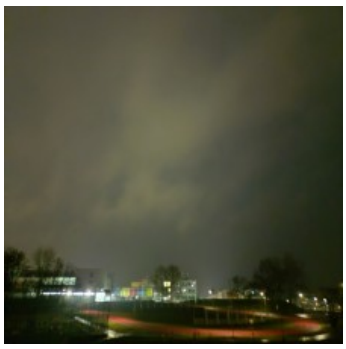
8:00(298.9,108.2)



11:00(280.1, 152.8),



17:00(489.1,286.7)



20:00(488.3,261.7)



23:00(131.7,107.0),



2:00(205.8,149.9)



8:00(146.1,201.0)



11:00(254.4,255.7)



14:00(333.4,153.5),



17:00(22.4,0.2)



20:00(47.3,1.4)



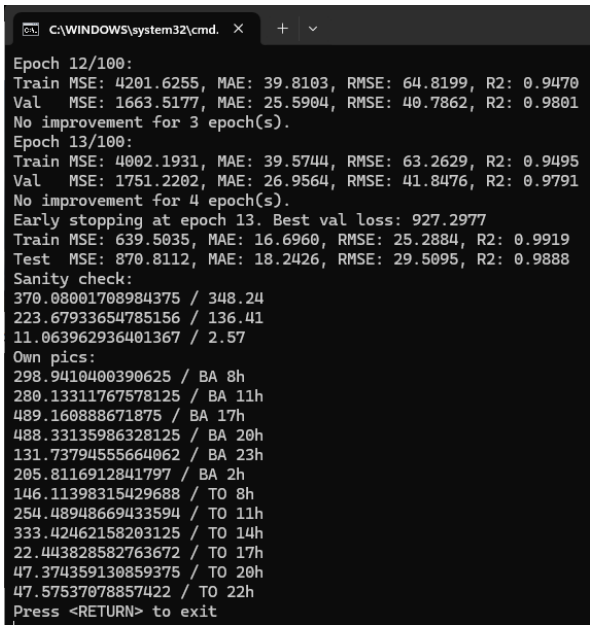
22:00(47.5,0.3)

Na obrázkoch BA 8:00 a 11:00 vidíme ako modely reagujú v hmle, na BA 17:00 a 20:00 vidíme ako modely kvantifikujú nočné osvetlenie, na BA 23:00 a BA 2:00 vidíme kvantifikáciu svetelného smogu.

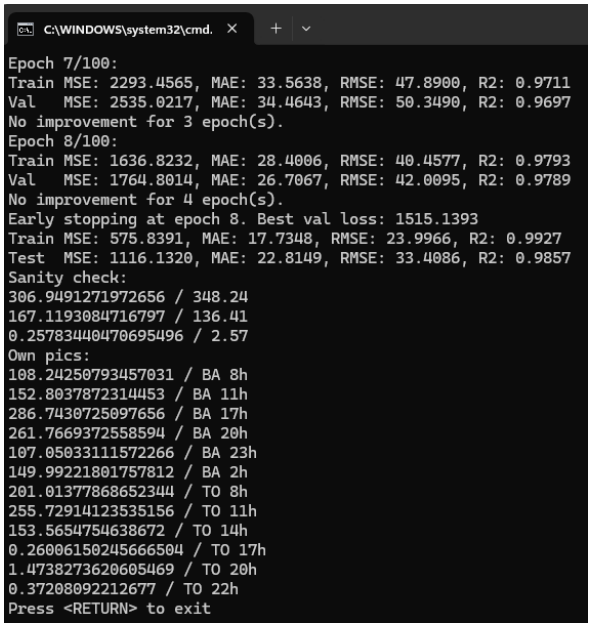
Na obrázkoch TO vidíme takmer perfektný prechod z nízkej hodnoty na vysokú na nízku.

Pocitovo sa javia hlavne pre obrázok TO 14:00 – R18+l4,fc❄ preds lepšie. Nižšie sú uvedené výpisy, z ktorých sa čerpalo.

Výpisy



```
C:\WINDOWS\system32\cmd.
Epoch 12/100:
Train MSE: 4201.6255, MAE: 39.8103, RMSE: 64.8199, R2: 0.9470
Val   MSE: 1663.5177, MAE: 25.5904, RMSE: 40.7862, R2: 0.9801
No improvement for 3 epoch(s).
Epoch 13/100:
Train MSE: 4002.1931, MAE: 39.5744, RMSE: 63.2629, R2: 0.9495
Val   MSE: 1751.2202, MAE: 26.9564, RMSE: 41.8476, R2: 0.9791
No improvement for 4 epoch(s).
Early stopping at epoch 13. Best val loss: 927.2977
Train MSE: 639.5035, MAE: 16.6960, RMSE: 25.2884, R2: 0.9919
Test  MSE: 870.8112, MAE: 18.2426, RMSE: 29.5095, R2: 0.9888
Sanity check:
370.08001708984375 / 348.24
223.67933654785156 / 136.41
11.063962936401367 / 2.57
Own pics:
298.9410400390625 / BA 8h
280.13311767578125 / BA 11h
489.160888671875 / BA 17h
488.33135986328125 / BA 20h
131.73794555664062 / BA 23h
205.8116912841797 / BA 2h
146.11398315429688 / TO 8h
254.48948669433594 / TO 11h
333.42462158203125 / TO 14h
22.443828582763672 / TO 17h
47.374359130859375 / TO 20h
47.57537078857422 / TO 22h
Press <RETURN> to exit
```



```
C:\WINDOWS\system32\cmd.
Epoch 7/100:
Train MSE: 2293.4565, MAE: 33.5638, RMSE: 47.8900, R2: 0.9711
Val   MSE: 2535.0217, MAE: 34.4643, RMSE: 50.3490, R2: 0.9697
No improvement for 3 epoch(s).
Epoch 8/100:
Train MSE: 1636.8232, MAE: 28.4006, RMSE: 40.4577, R2: 0.9793
Val   MSE: 1764.8014, MAE: 26.7067, RMSE: 42.0095, R2: 0.9789
No improvement for 4 epoch(s).
Early stopping at epoch 8. Best val loss: 1515.1393
Train MSE: 575.8391, MAE: 17.7348, RMSE: 23.9966, R2: 0.9927
Test  MSE: 1116.1320, MAE: 22.8149, RMSE: 33.4086, R2: 0.9857
Sanity check:
306.9491271972656 / 348.24
167.1193084716797 / 136.41
0.25783440470695496 / 2.57
Own pics:
108.24250793457031 / BA 8h
152.8037872314453 / BA 11h
286.7430725097656 / BA 17h
261.7669372558594 / BA 20h
107.05033111572266 / BA 23h
149.99221801757812 / BA 2h
201.01377868652344 / TO 8h
255.72914123535156 / TO 11h
153.5654754638672 / TO 14h
0.26006150245666504 / TO 17h
1.4738273620605469 / TO 20h
0.37208092212677 / TO 22h
Press <RETURN> to exit
```

```
print('Sanity check:')
print(prir('ins\\in_small_png\\Alpnach_20180101_11-00-00_ExposureStack_Image_10_image.png', model, device),'/ 348.24')
print(prir('ins\\in_small_png\\Alpnach_20180101_10-45-00_ExposureStack_Image_10_image.png', model, device),'/ 136.41')
print(prir('ins\\in_small_png\\Alpnach_20180101_15-45-00_ExposureStack_Image_10_image.png', model, device),'/ 2.57')
print('Own pics:')
print(prir('ins\\in_own_png\\20251204_080000465.JPG', model, device),'/ BA 8h')
print(prir('ins\\in_own_png\\20251204_110054501.JPG', model, device),'/ BA 11h')
print(prir('ins\\in_own_png\\20251204_170030174.JPG', model, device),'/ BA 17h')
print(prir('ins\\in_own_png\\20251204_200640866.JPG', model, device),'/ BA 20h')
print(prir('ins\\in_own_png\\20251204_230014408.JPG', model, device),'/ BA 23h')
print(prir('ins\\in_own_png\\20251205_020424642.JPG', model, device),'/ BA 2h')
print(prir('ins\\in_own_png\\20251204_075611.jpg', model, device),'/ TO 8h')
print(prir('ins\\in_own_png\\20251204_110046.jpg', model, device),'/ TO 11h')
print(prir('ins\\in_own_png\\20251204_135948.jpg', model, device),'/ TO 14h')
print(prir('ins\\in_own_png\\20251204_170216.jpg', model, device),'/ TO 17h')
print(prir('ins\\in_own_png\\20251204_194359.jpg', model, device),'/ TO 20h')
print(prir('ins\\in_own_png\\20251204_215339.jpg', model, device),'/ TO 22h')
```

Ďalšie obrázky je možné zapájať pridávaním do priečinka *ins\in_own_png* a obdobným príkazom na tomto mieste v kóde. Dodržať rozmer 224x224 zopnúť doprep kvartet 0000/1000

Záver

Program má 5 režimov podľa zopnutia do_r18, prep_misc, do_univar, do_bivar:
0000->úl.2+bonus vlastným regresorom, 1000->úl.4+bonus R18+l4,fc❄ regresorom
x100->preskúma bivariátnu koreláciu a nechá prostredie sprístupnené na negrafickú analýzu
x010->zobrazí univariátnu mapu
x001->zobrazí bivariátnu mapu
V práci sme preskúmali dataset, natrénovali a predviedli na ňom vlastnú CNN, dotrénovali odmrazené vrstvy ResNet18 a predviedli ResNet18 ako celok a otestovali obe CNN na vlastnoručne získaných obrázkoch.
V práci bol zapracovaný kód **cvicenie_1.py**, **cvicenie_2.py** a **customCNN.py**, ktorý bol prevzatý a adaptovaný, autormi sú učitelia predmetu I-SUNS v 2025/2026 na FEI STU BA. Zdroj na workflow pri EDA je https://medium.com/data-science/semi-automated-exploratory-data-analysis-eda-in-python-7f96042c9809 a https://www.dasca.org/newsroom/a-comprehensive-guide-to-mastering-exploratory-data-analysis (už neexistuje).

Použitie AI

AI bolo použité vo väčšej miere na generáciu funkcie **priebeh** a **errvis**. V práci bolo v strednej miere AI použité na **transformáciu customCNN na regresiu**, hľadanie bugov vo fázach transformácie, vysvetlenie týchto fáz, porovnávanie všeobecných postupov súvisiacich s regresiou a menšie záverečné ladenie na základe týchto postupov. V práci bolo použité AI v menšom meradle na generáciu a opravu kódu súvisiaceho s **jednoduchšími EDA grafmi na konci kódu**.